Sieci Neuronowe
Laboratorium
Programowanie w Pythonie - Materiały dodatkowe

Andrzej Kordecki

Zakład Teorii Maszyn i Robotów
Instytut Techniki Lotniczej i Mechaniki Stosowanej
Wydział Mechaniczny Energetyki i Lotnictwa
Politechnika Warszawska
2026

1 Python

Python to język programowania wysokiego poziomu i ogólnego przeznaczenia o rozbudowanym pakiecie darmowych bibliotek. Pierwszą implementację Pythona stworzył Guido van Rossum w 1989 roku. Główną ideą przy tworzeniu Pythona była czytelność, klarowność i zwięzłością kodu źródłowego, co zostało wyrażone przez jego twórcę słowami:

> Prosta idea nadania ludziom pierwszeństwa nad maszynami jest podstawą filozofii stojącej za stworzeniem Pythona.

Python rozwijany jest jako projekt Open Source zarządzany przez Python Software Foundation. Więcej informacji o projekcie Python wraz z możliwością pobrania języka Pyhton dostępna jest na stronie:

`http://www.python.org`.

2 Wersje Pythona

Wersje Pythona oznaczone są odpowiednią numeracją:

Python A.B.C, np. 3.9.7

- A (major) to główny numer wersji - główne zmiany w języku, często w jego podstawach.
- B (minor) to numer wersji pomocniczej - mniej istotne zmiany zmian, zazwyczaj zwiększające jego funkcjonalność.
- C (micro) to numer wersji mikro – wydanie z poprawkami błędów.

W trakcie zajęcie będzie wykorzystywana wersja 3.X, która nie jest kompatybilna z wersją 2.X. Wersję języka Pyton można sprawdzić wpisując następujący kod programu:

```
import sys
print(f"Wersja Pythona: {sys.version[0:6]}")
```

Zalecana wersja Pythona do korzystania podczas zajęć to wersja 3.7 lub nowsza.

3 Typy danych i zmiennych

Język programowania Python jest językiem dynamicznie typowanym, co oznacza, że typy danych są określane automatycznie w trakcie wykonywania programu, a nie na etapie deklaracji zmiennych. Python zawiera wiele typów danych, które można podzielić do m.in. następujących kategorii:

- Liczbowe: Booleans (Wartości logiczne), Integers (Liczby całkowite), Floats (Liczby zmiennoprzecinkowe), complex,
- Sekwencji: Strings (Napisy), Lists (Listy), Tuples (Krotki),
- Odwzorowujące : Dictionaries (Słowniki), Sets (Zbiory)
- NoneType: Reprezentuje brak wartości lub pustą zmienną.
- Custom Objects (Własne obiekty): class (klasy).

Przy czym następowały zmiany w przynależności do określonego typu grupy danych w kolejnych wersja Pythona.

3.1 Dane liczbowe

W języku Python dostępne są dwa główne typy danych liczbowych: int (liczby całkowite) i float (liczby zmiennoprzecinkowe). W Pythonie typ danych jest ustawiany podczas przypisywania wartości do zmiennej, a przez to nie wymaga jego wcześniejszej deklaracji jak ma to miejsce w innych językach programowania:

```
x = 5
print(f"Typ {type(x)}, Wartosc = {x}")
# Wynik: Typ <class 'int'>, Wartosc = 5

y = 3.14
print(f"Typ {type(y)}, Wartosc = {y}")
# Wynik: Typ <class 'float'>, Wartosc = 3.14
```

Python umożliwia wykonywanie podstawowych operacji matematycznych, takich jak dodawanie, odejmowanie, mnożenie i dzielenie.

```
x, y = 2, 3
print(f"Dodawanie: {x + y}") # Wynik: 5
print(f"Odejmowanie: {x - y}") # Wynik: -1
print(f"Mnozenie: {x * y}") # Wynik: 6
print(f"Dzielenie: {x / y}") # Wynik: 0.6666
```

Python pozwala na obliczanie potęg i pierwiastków.

```
import math

x, y = 4, 2
print(f"Potegowanie: {x ** y}") # Wynik: 16
print(f"Pierwiastek: {math.sqrt(x)}") # Wynik: 2
```

Możesz zaokrąglać liczby zmiennoprzecinkowe i konwertować typy danych.

```
x = 3.75
y = round(x)  # Zaokraglenie do najblizszej calkowite
print(f"Zaokraglenie: {y}")

x = 10
print(f"Typ: {type(x)}") # Typ: <class 'int'>
y = float(x)  # Konwersja int na float
print(f"Wynik: {y}, {type(y)}") # Wynik: 10.0, <class 'float'>

x = 9.87
print(f"Typ: {type(x)}") # Typ: <class 'float'>
y = int(x)    # Konwersja float na int
print(f"Wynik: {y}, {type(y)}") # Wynik: 9, <class 'int'>
```

Python umożliwia wykonywanie operacji logiki Boole'a, ale używa angielskich słów zamiast symboli do ich realizacji:

```
t, f  = True, False
print(t and f) # Wynik: "False"
print(t or f)  # Wynik: "True"
print(not t)   # Wynik: "False"
print(t == f)  # Wynik: "False"
print(t != f)  # Wynik: "True"
```

3.2 Dane tekstowe

Ciągi znaków tekstowych, czyli typ string, są definiowane przez otoczenie tekstu pojedynczymi ('tekst') lub podwójnymi ("tekst") cudzysłowami. Do odwołania się do poszczególnych elementów stringa należy wykorzystać instrukcje stosowane do typu listy (omówione później). Przykłady tworzenia danych tekstowych:

```
hello = 'hello'
world = "world"
print(hello)       # Wynik: "hello"
txt = hello + ' ' + world
print(txt)  # Wynik: "hello world"
print(txt[2:6])  # Wynik: "llo"
```

Format string jest często stosowany do wyświetlania otrzymanych wyników. Istnieje kilka sposobów wyświetlania wartości zmiennych:

```
print('%f %s %s %d' % (3.14, hello, world, 0))
print('{} {} {} {}'.format(3.14, hello, world, 0))
print(f'{3.14} {hello} {world} {0}')
# Wynikiem dla wszystkich opcji jest: "3.14 hello world"
```

Dokładność wyświetlania wartości liczbowych można regulować:

```
print('{:f}'.format(3.141592653)) # Wynik: "3.141593"
print('{:.2f}'.format(3.141592653)) # Wynik: "3.14"
print('{:06.2f}'.format(3.141592653)) # Wynik: "003.14"
```

3.3 Dane sekwencyjne - listy

Istnieją 4 wbudowane typy danych w Pythonie używanych do przechowywania kolekcji danych: List, Tuple, Set i Dictionary, które różnią się w działaniu i sposobie przechowywania danych.

Listy służą do przechowywania wielu elementów w jednej zmiennej, czyli są odpowiednikiem tablicy (macierzy) w Pythonie. Lista może zawierać różne typy danych lub inne dane sekwencyjne (np. int, float, string, czy nawet list). Tworzymy listy za pomocą nawiasów kwadratowych, w których obejmowane są jej elementy. Cechy list:

- elementy listy są uporządkowane i indeksowane przez numery (uporządkowane),
- można modyfikować elementy listy (mutowalne),
- może zawierać wiele elementów różnych typów danych o tych samych wartościach (duplikaty).

Kiedy mówimy, że listy są uporządkowane, oznacza to, że elementy listy mają określoną kolejność i ta kolejność się nie zmienia. Nieuporządkowany typ danych oznacza, że elementy nie mają zdefiniowanej kolejności, nie można odwoływać się do elementu za pomocą indeksu. Przyporządkowanie indeksu elementom listy można wykonać:

- od przodu listy: pierwszy element listy z indeksem = 0, drugi element listy z indeksem = 1 itd.
- od końca listy: ostatni element listy z indeksem = -1, przedostatni element listy z indeksem = -2 itd.

Możemy odczytywać zawartość zarówno pojedynczych elementów listy lub od razu całego ciągu elementów. Elementy w liście są indeksowane, zaczynając od zera. Możesz uzyskać dostęp do elementu podając jego indeks w nawiasach kwadratowych. W celu odczytania ciągów elementów listy należy podać numer indeksu, od którego mają zostać odczytane elementy z listy oraz dwukropek. Przykład tworzenia listy, indeksowania i modyfikowania elementów listy:

```
xs = [3, 1, 2]
print(len(xs)) # Wynikem jest dlugosc listy: 3
print(xs, xs[2]) # Wynik: "[3, 1, 2] 2"
print(xs[-1]) # Wynik to wartosc ostatniego indeksu: "2"
# Wycinanie (slicing) listy:
print(xs[:-1]) # Wynik: "[3, 1]"
print(xs[0:2]) # Wynik: "[3, 1]"

# Przypisanie innego typu danych do elementu listy
xs[2] = 'world'
print(xs)        # Wynik: "[3, 1, 'world']"
```

Dodawanie i usuwanie elementów poszczególnych elementów listy:

```
xs = [3, 1, 2]

# Dodanie elementu na koncu listy
xs.append('tree')
print(xs) # Wynik: "[3, 1, 2, 'tree']"
# Mozesz rowniez wstawic element na konkretnej pozycji
xs.insert(1, 23)  # Wstawia 5 na pozycje o indeksie 1
print(xs) # Wynik: [3, 23, 1, 2, 'tree']
# Usuniecie ostatniego elementu
xs.pop() # rownoznaczne z xs.pop(3)
print(xs) # Wynik: "[3, 23, 1, 2]"
# Usuniecie 3-ciego elementu
xs.remove(3)
print(xs) # Wynik: "[23, 1, 2]"
```

Operacje matematyczne z wykorzystaniem kilku list związane są operacjami na zbiorach, a nie elementach listy. Stąd, dodanie listy nie spowoduje dodania wartości dwóch odpowiadających sobie elementów list, ale połączenie obu zbiorów. W celu operacji matematycznych na macierzach należy posłużyć się biblioteką numpy.

```
x1, x2 = [3, 1, 2, 12, 1], [34, 21, 42]

# laczenie list (konkatenacji listy)
x1 = x1 + x2
print(x1) # Wynik: "[3, 1, 2, 12, 1, 34, 21, 42]"

# Wynik operacji moze byc inny od oczekiwanego
x2 = x2*2
print(x2) # Wynik: "[34, 21, 42, 34, 21, 42]"
```

Przypisania nowej zmiennej nie zawsze tworzy jego kopie (płytkie i głębokie kopiowanie). W zależności od sposobu przypisania wartości jednej zmiennej, będzie przypisana kopia wartości albo wskaźnik do pamięci z zawartością danej zmiennej.

```
x = [95, 100, 105]

x_1 = x # przekazuje wskaznik do listy
x_2 = list(x) # plytkie kopiowanie
x_3 = x.copy() # plytkie kopiowanie

x[:] = [102, 111, 143, 199]

print(x_1) # Prints "[102, 111, 143, 199]"
print(x_2) # Prints "[95, 100, 105]"
print(x_3) # Prints "[95, 100, 105]"
```

3.4 Krotki (Tuple)

Krotka (Tuple) jest uporządkowaną kolekcją elementów, której zawartości nie można zmieniać po jej utworzeniu. Krotki najczęściej tworzy się za pomocą nawiasów okrągłych. Cechy krotek:

- elementy krotki są uporządkowane i dostępne za pomocą indeksów,
- nie można dodawać, usuwać ani zastępować elementów krotki po jej utworzeniu (krotka jest niemutowalna),
- krotka może zawierać elementy różnych typów danych,
- krotka może zawierać powtarzające się wartości (duplikaty).

Należy pamiętać, że niemutowalność krotki dotyczy jej struktury. Jeżeli krotka zawiera obiekt mutowalny, np. listę, zawartość tego obiektu może zostać zmieniona. Zastosowanie krotek:

- przechowywanie danych, których liczba i kolejność nie powinny się zmieniać,
- zwracanie kilku wartości z funkcji,
- używanie jako kluczy w słownikach lub jako elementów zbiorów, pod warunkiem że wszystkie elementy krotki są niemutowalne.

Przykładowe operacje na krotkach:

```
t = (1, [5, 3], 'abc')

print(type(t))    # Wynik: <class 'tuple'>
print(t)          # Wynik: (1, [5, 3], 'abc')
print(t[1])       # Wynik: [5, 3]
print(t[1][0])    # Wynik: 5

# Nie mozna zmienic elementu krotki:
t[0] = 10      # Blad

# Mozna jednak zmienic zawartosc listy znajdujacej sie w krotce:
t[1][0] = 10
print(t)          # Wynik: (1, [10, 3], 'abc')
```

3.5 Słowniki (Dictionary)

Słowniki (Dictionary) jest kolekcją danych przechowującą elementy w postaci par (klucz: wartość). Od wersji Python 3.7 słowniki zachowują kolejność dodawania elementów. Klucze słownika muszą być unikalne i należeć do typów niemutowalnych. Cechy słowników:

- elementy słownika są indeksowane za pomocą kluczy, a nie indeksów liczbowych,
- słownik jest strukturą mutowalną – można dodawać, usuwać i zmieniać jego elementy,
- klucze muszą być unikalne,
- wartości przypisane do kluczy mogą się powtarzać,
- wartości mogą być dowolnego typu danych.

Tworzenie słownika i podstawowe operacje:

```
d = {'car': 'small', 'truck': 'big'}

print(d['car'])       # Wynik: small
print('car' in d)     # Wynik: True

# Dodanie nowego elementu
d[1] = 25
print(d)
# Wynik: {'car': 'small', 'truck': 'big', 1: 25}

# Zmiana wartosci istniejacego elementu
d['car'] = 'medium'
print(d['car'])       # Wynik: medium

# Usuniecie elementu
del d['car']
print(d)
# Wynik: {'truck': 'big', 1: 25}
```

3.6 Zbiory (Set)

Zbiory (Set) jest nieuporządkowaną kolekcją unikalnych elementów. Zbiory są szczególnie przydatne wtedy, gdy istotne jest szybkie sprawdzanie przynależności elementów do kolekcji lub usuwanie powtarzających się wartości. Zbiór można utworzyć za pomocą nawiasów klamrowych lub funkcji set(). Cechy zbiorów:

- każdy element zbioru może wystąpić tylko jeden raz – zbiory nie zawierają duplikatów,
- elementy zbioru nie są indeksowane i nie można odwoływać się do nich za pomocą numerów indeksów,
- zbiór jest mutowalny – można dodawać i usuwać jego elementy,
- elementami zbioru mogą być wyłącznie obiekty niemutowalne, np. liczby, łańcuch znaków lub odpowiednie krotki,
- kolejność elementów zbioru nie powinna być traktowana jako gwarantowana.

Jednym z częstych zastosowań zbiorów jest usuwanie duplikatów z listy:

```
lista = [3, 5, 3, 2, 5, 7, 2, 2]

unikalne = set(lista)
print(unikalne) # Kolejnosc elementow nie jest gwarantowana
```

4 Narzędzia kontroli przepływu

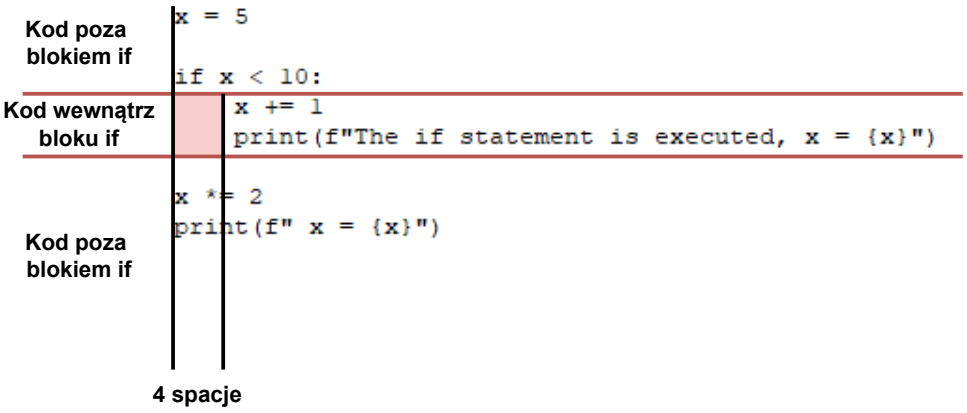
Interpreter Pythona wykonuje kod programu wiersz po wierszu, ale możemy to zmienić za pomocą narzędzi kontroli przepływu. Narzędzia te wpływają na kolejność wykonywania kodu w programie. Przykład najczęściej używanych narzędzi przepływu:

- if
- for i while
- def
- inne

Definicje funkcji, pętli i instrukcji warunkowych mają podobną składnię i wykorzystują wcięcia do wykonywania przypisanego im bloku kodu programu poprzedzonych znakiem dwukropka.

Python używa **białych znaków** (4 spacji w PEP 8) do wyznaczenia zasięgu działania instrukcji.

Instrukcje mogą być zagnieżdżone na kilka poziomów poprzez wykorzystanie wielokrotności 4 spacji. Przykład bloku instrukcji „if”:



Odstępy w liniach kodu mają duże znaczenie w jego interpretacji. Dlatego podkreślam, NIE używamy jak w innych językach programowania nawiasów lub słów, ale białych znaków do wyznaczenia zasięgu działania instrukcji.

4.1 Instrukcja if

Za pomocą instrukcji warunkowej ’if’ możemy tworzyć kod, który w zależności od postawionego warunku się wykona albo nie. W instrukcji warunkowej możemy dodatkowo używać instrukcji:

- elif - jeśli poprzednie warunki nie były spełnione, wypróbuj ten warunek,
- else - wykonuje ten warunek jeżeli żaden warunek wcześniej nie został spełniony

Przykład działania:

```
# Wczytanie danych z klawiatury
x = int(input("Please enter an integer: "))

if x < 0:
  print('Less than zero')
elif x == 0:
  print('Zero')
elif x == 1:
  print('Single')
else:
```

```
  print('More')

# Wynika dla x = 0: 'Zero'
# Wynika dla x = 1: 'Single'
# Wynika dla x = -1: 'Less than zero'
# Wynika dla x = 2: 'More'
```

4.2 Instrukcje for i while

Instrukcja for różni się w jej działaniu od innych języków programowania. Nie przechodzi w kolejnych iteracjach od zadanej wartości początkowej do warunku zatrzymania z określonym krokiem, ale wykonuje iteracje po kolejnych elementach jakiejkolwiek sekwencji (np. listy) zgodnie z ich uporządkowaniem. Przykład działania:

```
words = ['cat', 'dog', 'bird']
for w in words:
  print(f"word w = {w}, length = {len(w)}")

# Wynik:
# word w = cat, length = 3
# word w = dog, length = 3
# word w = bird, length = 4
```

Python umożliwia również w łatwy sposób iterowanie po sekwencji z liczb za pomocą funkcji range().

```
# Sekwencja od 2 do 6 z krokiem 2
print(list(range(2, 6, 2))) # Wynik: [2, 4]
# Sekwencja od 3 do 10 z krokiem 4
print(list(range(3, 10, 4))) # Wynik: [3, 7]

for w in range(5):
  print(f"w = {w}")
# Wynik: #w = 0 #w = 1 #w = 2 #w = 3 #w = 4
```

Instrukcja while jest używana do wielokrotnego wykonywania bloku kodu, dopóki warunek jest prawdziwy (True).

```
i = 1
while i < 6:
  print(i)
  i += 1
# Wynik: #w = 1 #w = 2 #w = 3 #w = 4 #w = 5
```

Słowa kluczowe 'continue' i 'break' umożliwiają wpływ na kolejność wykonywanych instrukcji w pętli 'while' oraz 'for'. Instrukcje te mogą się pojawić tylko w miejscu, które składniowo jest zagnieżdżone funkcjach pętli:

- continue - powoduje przejście do następnego cyklu iteracji najściślej otaczającej ją pętli,
- break - powoduje przerwanie wykonania iteracji i wyjście z pętli.

Przykład zastosowania:

```
for num in range(1, 4):
  if num % 2 == 0:
    n += 1
    print("Liczba parzysta", num)
    continue
  print("Liczba nieparzysta", num)
# Wynik:
# Liczba nieparzysta 1
# Liczba parzysta 2
# Liczba nieparzysta 3
```

4.3 Funkcje

Funkcja to blok kodu, który działa tylko wtedy, gdy jest wywoływana. Możemy przekazać dane (znane jako argumenty lub parametry funkcji) do funkcji i możemy w rezultacie wykonania bloku kodu zwrócić dane. Słowo kluczowe 'def' wprowadza definicję funkcji, po której wpisujemy nazwę funkcji i dodatkowo w nawiasach okrągłych możemy zdefiniować listę parametrów.

```
def potegowanie(liczba, potega):
  wynik = liczba # zmienna lokalna
  for i in range(1, potega):
    wynik = wynik*liczba
  return wynik

l, p = 3, 2 # zmienne globalne
print(potgowanie(l, p))  # Wynik: 9
print(potgowanie(5, 3))  # Wynik: 125
```

Wykonywanie poleceń na zmiennych w bloku kodu wymaga określenia ich podziału ze względu na zasięg działania:

- zmienne globalne - zmienne zdefiniowane poza funkcją. Dostęp do zmiennej globalnej mamy wewnątrz, jak i na zewnątrz funkcji.
- zmienne lokalne - zmienne zdefiniowane wewnątrz funkcji. Dostęp do zmiennej lokalnej możemy uzyskać tylko wewnątrz funkcji.

Mimo że definiujemy zmienną o tej samej nazwie wewnątrz i na zewnątrz funkcji, zmiana wartości zmiennej w zakresie lokalnym nie wpływa na wartość zmiennej w zakresie globalnym. Istnieje możliwość obejścia tej zasady przez użycie słowa kluczowego 'global' przed zmienną, abyśmy mogli zmienić zmienną na poziomie globalnym

```
def moja_funkcja():
  global zmienna
  print(zmienna)
  zmienna='zasieg lokalny funkcji'

zmienna ='zmienna lokalna'
moja_funkcja()
print(zmienna)
# Wynik:
# zmienna lokalna
# zasieg lokalny funkcji
```

Parametry funkcji mogą mieć domyślne wartości oraz możemy przypisywać im wartości przez ich słowa kluczowe (nazwy).

```
def fnieliniowa(liczba1=1, liczba2=2, liczba3=3):
  return liczba1*liczba2 + liczba3

print(fnieliniowa())              # Wynik: 5
print(fnieliniowa(2, liczba3=6))  # Wynik: 10
```

Symbol 'lambda' umożliwia utworzenie jednolinijkowej anonimowej funkcji. Tworzenie funkcji zaczynamy od słowa 'lambda', następnie podajemy listę parametrów przyjmowanych przez funkcje i kończymy definicję za pomocą znaku „:" po którym definiujemy zawartość funkcji.

```
def fun1(a, b, c = 1):
  return a + b - c

fun2 = lambda a, b, c = 1: a + b - c
print(f"fun1 = {fun1(1, 2, 2)} = fun2 = {fun2(1, 2, 2)}")
# Wynik: "fun1 = 1 = fun2 = 1"

def count_c(n, m):
  return lambda a: a*n+m

d1 = count_c(5, 5)
print(f"d1 = {d1(2)}") # Wynik: "d1 = 15"
d2 = count_c(5, 5)(2)
print(f"d2 = {d2}") # Wynik: "d2 = 15"
```

4.4 List comprehension

List comprehension to skrócony sposób tworzenia nowych list na podstawie istniejących kolekcji danych. Pozwala zastąpić prostą pętlę "for", często połączoną z instrukcją warunkową "if", pojedynczym wyrażeniem. Ogólna postać tworzenia listy za pomocą list comprehension jest następująca:

```
nowa_lista = [wyrazenie for element in kolekcja]
```

Przykładowo, listę zawierającą kwadraty liczb można utworzyć za pomocą klasycznej pętli:

```
liczby = [1, 2, 3, 4, 5]
kwadraty = []

for liczba in liczby:
  kwadraty.append(liczba**2)

print(kwadraty)
# Wynik: [1, 4, 9, 16, 25]
```

To samo można zapisać krócej za pomocą list comprehension:

```
liczby = [1, 2, 3, 4, 5]

kwadraty = [liczba**2 for liczba in liczby]

print(kwadraty)
# Wynik: [1, 4, 9, 16, 25]
```

Do wyrażenia można również dodać warunek określający, które elementy mają znaleźć się w nowej liście. Ogólna postać takiej konstrukcji jest następująca:

```
nowa_lista = [wyrazenie for element in kolekcja if warunek]
```

Przykładowo, listę zawierającą wyłącznie liczby parzyste można utworzyć w następujący sposób:

```
liczby = [1, 2, 3, 4, 5, 6]

parzyste = [liczba for liczba in liczby if liczba % 2 == 0]

print(parzyste)
# Wynik: [2, 4, 6]
```

Możliwe jest jednoczesne przekształcanie elementów oraz ich filtrowanie. Przykładowo można utworzyć listę kwadratów wyłącznie dodatnich liczb:

```
liczby = [-3, -1, 0, 2, 4]

kwadraty_dodatnich = [liczba**2 for liczba in liczby if liczba > 0]

print(kwadraty_dodatnich)
# Wynik: [4, 16]
```

List comprehension można również wykorzystać razem z instrukcją warunkową `if-else`. W takim przypadku warunek umieszczany jest przed pętlą "for":

```
liczby = [1, 2, 3, 4, 5]

opis = ["parzysta" if liczba % 2 == 0 else "nieparzysta" for liczba in liczby]

print(opis)
# Wynik:
# ['nieparzysta', 'parzysta', 'nieparzysta', 'parzysta', 'nieparzysta']
```

Należy zwrócić uwagę na różnicę między dwoma zastosowaniami instrukcji "if":

- warunek zapisany na końcu wyrażenia służy do **filtrowania elementów**:

  ```
  [x for x in liczby if x > 0]
  ```

- konstrukcja `if-else` zapisana przed pętlą `for` służy do **wyboru wartości**, która zostanie dodana do listy:

  ```
  [x if x > 0 else 0 for x in liczby]
  ```

List comprehension pozwala tworzyć bardziej zwięzły kod, jednak nie zawsze jest najlepszym rozwiązaniem. W przypadku bardziej złożonych operacji klasyczna pętla "for" może być bardziej czytelna i łatwiejsza do zrozumienia. Ten sam problem można więc często rozwiązać zarówno za pomocą zwykłej pętli, jak i list comprehension.

4.5 Klasy

Programowanie obiektowe (Object-oriented programming, OOP) łączy grupę atrybutów (danych) z funkcjami lub metodami w jednostkę zwaną 'obiektem'. Napisany obiektowo program zawiera zbiór obiektów, komunikujących się pomiędzy sobą w celu wykonywania zadań. Podejście to różni się od tradycyjnego programowania proceduralnego, gdzie dane i procedury nie są ze sobą bezpośrednio związane. Podstawowe pojęcia OOP:

- Klasa – zawiera definicje formatu danych oraz procedur dla danego obiektu.
- Obiekt – instancja klasy.

Każdy obiekt jest instancją konkretnej klasy np. obiekt 'pies' może być instancją klasy 'zwierzęta'. Procedury w programowaniu obiektowym nazywane są metodami, a zmienne: obszarami, członkami, atrybutami albo właściwościami. Programowanie obiektowe zapewnia warstwę abstrakcji, która może być użyta do odseparowania wewnętrznego od zewnętrznego kodu programu. Obiekty są wywoływane przez specjalne typy metod w klasie znanych jako konstruktory. Program może stworzyć wiele instancji tej samej klasy w czasie działania, które działają niezależnie od siebie.

Klasę definiujemy przez słowo kluczowe 'class', a konstruktor przez '__init__'. Konstruktor jest wywoływany podczas inicjalizacji nowego obiektu, ale nie jest on obowiązkowy przy definiowaniu klasy (opcjonalny). Konstruktor służy do ustawienia różnych wartości i przygotowania parametrów/danych obiektu do naszych celów. Przy definiowaniu funkcji klasy często będziemy posługiwać się słowem kluczowym 'self' w miejscu pierwszego parametru. Używając słowa kluczowego „self", które reprezentuje instancję klasy, możemy uzyskać dostęp do atrybutów i metod klasy w Pythonie. Natomiast obiekt definiujemy przez nazwę wcześniej stworzonej klasy. Przykład definicji klasy i tworzenie obiektu (instancji):

```
# Definija klasy
class MyClass:
  color = ["red"] #zmienne sekwencyjne klasy sa wspoldzielone
  kcolor = 'yellow'

  def __init__(self, id=55):
    self.id = id # nowa instancja atrybutu
    self.icolor = ['red']

  def myinfo(self):
    print(f"Hello {self.id}")

# Definija obiektow
k1 = MyClass(89)
k2 = MyClass()

# Zmiana atrybutow
k1.color[0] = 'green'
k1.icolor[0] = 'green'
k1.kcolor = 'green'
print(f'Zmienna klasy sekwencyjna {k2.color} i niesekwencyjna {k2.kcolor},
zmienna instancji  {k2.icolor}')
# Wynik: 'Zmienna klasy sekwencyjna ['green'] i niesekwencyjna yellow,
#        zmienna instancji  ['red']'

k1.myinfo() # Wynik "Hello 89"
print(k1.id) # Wynik "89"
```

Paradygmaty programowania obiektowego:

- Dziedziczenie (inheritance),
- Abstrakcja (abstraction),
- Polimorfizm (polymorphism),
- Hermetyzacja (encapsulation).

4.6 Dziedziczenie

Dziedziczenie to procedura, w której jedna klasa dziedziczy atrybuty i metody innej klasy. Klasy zazwyczaj są zorganizowane w hierarchie, gdzie klasa może mieć jedną lub więcej klas nadrzędnych lub podrzędnych, np. figury geometryczne: każda figura ma pole i obwód, a w przypadku koła można dodać promień, a kwadratu długość

boku. W przypadku stworzenie klasy dla każdej figury musielibyśmy wykorzystać ten sam kod wiele razy, a zmiany w trakcie rozwoju programu musiałby być ponownie przepisane do każdej klasy. Dlatego zaproponowano dziedziczenie jako rozwiązanie tego problemu. Polega ono na tym, że tworzymy jedną klasę (bazową), która ma pewne cechy, a następnie tworzymy drugą klasę, która po niej dziedziczy te cechy:

- Klasa nadrzędna, której właściwości i metody są dziedziczone (Parent).
- Klasa podrzędna, która dziedziczy właściwości z klasy nadrzędnej (Child).

Klasa potomna może mieć własne właściwości i metody. Składnia:

```
# Klasa Parent
class parent_class:
  pass

# Klasa Child, ktora dziedziczy z klasy parent_class
class child_class(parent_class):
  pass
```

Przykład zastosowania:

```
class Car:  # klasa Parent
  def __init__(self, name, model):
    self.name = name
    self.model = model

  def informations(self):
    return f"Car brand {self.name}, model {self.model}"

class Opel(Car):     # klasa Child
  pass

class Audi(Car):     # klasa Child
  def audi_info(self):
    return "Audi class description."

car1 = Opel("Opel","Corsa")
print(car1.informations()) # Wynik: "Car brand Opel, model Corsa"

car2 = Audi("Audi","A8")
print(car2.informations()) # Wynik: "Car brand Audi, model A8"
print(car2.audi_info())    # Wynik: "Audi class description.""
```

4.7 Polimorfizm

Polimorfizm oznacza posiadanie wielu form (po grecku „poly" – wiele i „morf" – formy) i w programowaniu obiektowym oznacza możliwość używania tej samej nazwy metody lub funkcji w różnych kontekstach, przy czym implementacja zależy od konkretnej klasy lub typu danych. Realizacja polimorfizmu wykonywana jest poprzez dziedziczenie i następnie dynamiczne wiązanie metod wraz dodaniem własnej implementacji. Aby wykorzystać polimorfizm:

- Klasa dziedzicząca może być używana wszędzie tam, gdzie jest oczekiwana klasa bazowa.
- Instancja klasy dziedziczącej jest traktowana jako instancja klasy bazowej, ale może nadpisywać jej metody, dostarczając własną implementację.

Przykład zastosowania:

```
class Car:  # klasa Parent
  def description(self):   # ta sama nazwa funkcji
    return f"Car brand {self.name}, model {self.model}"

class Opel(Car): # klasa Child
  def description(self):   # ta sama nazwa funkcji
    print("Opel class informations.")

class Audi(Car): # klasa Child
  def description(self):   # ta sama nazwa funkcji
    print("Audi class informations.")

ocar, acar = Opel(), Audi()
```

```
for car in (ocar, acar):
car.description()
# Wynik:
# Opel class informations.
# Audi class informations.
```

4.8 Abstrakcja

Abstrakcji używamy do ukrywania wewnętrznych szczegółów lub implementacji funkcji i pokazywania tylko jej funkcjonalności:

- Metoda abstrakcyjna jest metodą zadeklarowaną, ale nie zawiera implementacji.
- Każda klasa z przynajmniej jedną funkcją abstrakcyjną jest klasą abstrakcyjną.
- Nie możemy stworzyć obiektu dla klasy abstrakcyjnej.

Przykład zastosowania:

```
from abc import ABC, abstractmethod

class Shape(ABC):
  @abstractmethod
  def area(self):
    pass

class Rectangle(Shape):
  def __init__(self, x,y):
    self.l = x
    self.b = y

  def area(self):
    return self.l*self.b

r = Rectangle(10,20)
print ('area: ',r.area()) # Print "200"
```

4.9 Hermetyzacja

Hermetyzacja (enkapsulacja) polega na ukrywaniu pewnych danych składowych lub metod obiektów danej klasy tak, aby były one dostępne tylko metodom wewnętrznym danej klasy lub funkcjom zaprzyjaźnionym. Główne zalety stosowania hermetyzacji:

- Ukrywanie szczegółów implementacji – użytkownik klasy nie musi znać szczegółów jej wewnętrznej budowy do jej zastosowania.
- Bezpieczeństwo danych – chroni przed niepożądanymi modyfikacjami jakie może wprowadzić użytkownik w wewnętrzne parametry lub metody klasy.
- Łatwość utrzymania kodu – zmiany w implementacji klasy nie wpływają na kod korzystający z jej publicznego interfejsu.

Zazwyczaj w językach obiektowych wyróżniamy trzy klasy dostępności danych i funkcji klasy:

- publiczne – dostęp mają wszyscy,
- chronione – dostęp mają klasy dziedziczące,
- prywatne – dostęp ma tylko dana klasa.

Dodanie jednego ('_') lub dwóch podkreśleń ('__') na początku powoduje, że zmienna lub funkcja jest traktowana odpowiednio jako chroniona i prywatna. Przykład zastosowania:

```
class BankAccount:
  def __init__(self, balance):
    self.__balance = balance  # Prywatny atrybut

  def deposit(self, amount):
    if amount > 0:
```

```
      self.__balance += amount
      return f"Deposited: {amount}"
    return "Invalid amount"

  def get_balance(self):
    return self.__balance  # Bezposredni dostep do atrybutu jest niemozliwy, ale mamy metode
    dostepowa

# Przyklad uzycia
account = BankAccount(1000)
print(account.deposit(500))
print(account.get_balance())

# Proba bezposredniego dostepu do prywatnej zmiennej zwroci AttributeError
# print(account.__balance)
```

5 Biblioteki w języku Python

Biblioteka w języku Python to w dużym uproszczeniu zbiór funkcji i metod, które stworzono w celu ułatwienia implementacji rozwiązań. Elementy procesu przetwarzania danych, które często są stosowane mają gotową implementacje w bibliotekach. Stanowi to duże uproszczenie w pisaniu programu i implementacji różnych algorytmów. Często metoda, którą chcemy zaimplementować, została już napisana i udostępniona w postaci biblioteki. Istnieje wiele rodzajów bibliotek od bardziej ogólnego charakteru związanego z operacjami matematycznymi, czy tworzeniem wizualizacji przebiegów zmiennych do wysoce wyspecjalizowanych związanych konkretnymi zadania związanych z wizualizacji obrazów geodezyjnych. Warto zauważyć, że samo zainstalowanie Pythona udostępni zestaw modułów o różnym zastosowaniu tworzących Bibliotekę Standardową Pythona (ang. The Python Standard Library). Każda porządna biblioteka powinna posiadać dokumentację z objaśnia możliwości wykorzystania zawartych w niej funkcji i metod.

Instalacja biblioteki w systemie Python następuje w linii poleceń systemu (ważne jest ustawienie ścieżek w systemie). Poniżej został przedstawiony typowy kod instalacji biblioteki, ale należy zwrócić uwagę, że każda biblioteka może mieć unikalne nazewnictwo. Stąd, warto przed wpisaniem polecenia sprawdzić instrukcje instalacji zamieszczoną na stronie projektu danej biblioteki.

```
pip install opencv-python
pip install matplotlib
```

Zastosowanie biblioteki w systemie Python jest możliwe za pomocą słowa kluczowego "import". Istnieje możliwość zaimportowania całej biblioteki lub tylko wybranych jej modułów/funkcji z wykorzystaniem słowa kluczowego "from" oraz skrócenia nazwy z wykorzystaniem słowa kluczowego "as".

```
import cv2
import numpy as np

from tensorflow import keras
from tensorflow.keras import layers
```

Warto zauważyć, że podczas zajęć laboratoryjnych będziemy korzystali z Google Colab, która posiada zainstalowanych wszystkie niezbędne do wykonania laboratoriów biblioteki.

5.1 Biblioteka Numpy

Biblioteka NumPy została stworzona, aby umożliwić szybkie i sprawne operacje na macierzach. Zasadniczo numpy oferuje szeroki zakres operacji na liczbach, a najważniejszą zaletą biblioteki jest szybkość działania. Operacja za pomocą funkcji biblioteki są znacznie szybsze, niż gdybyśmy chcieli te same operacje wykonać za pomocą list czy też pętli. Bibliotekę numpy importuje do kodu Pythona tak jak każdą inną bibliotekę:

```
  import numpy as np
```

Skrót np jest powszechnie stosowanym skrótem dla biblioteki numpy. Podstawowym typem danych, który otrzymujemy wraz z NumPy jest ndarray, czyli N-dimmentional array, czyli lista/tablica wielowymiarowa. Często w skrócie mówi się na to tablica, jednak nie można mylić z domyślą listą / tablicą w Python, która jest jedno wymiarowa.

Tablica numpy to macierz wartości z wszystkimi elementami tego samego typu, indeksowana przez nieujemne liczby całkowite:

```
a = np.array([1, 2, 3])
print(type(a))  # Prints "<class 'numpy.ndarray'>"
print(a.shape)  # Prints "(3,)"

# Change an element of the array
print(a[0], a[1], a[2])  # Print: 1 2 3
a[0] = 0
print(a)                 # Print: [0 2 3]

b = np.array([[1,2,3],[4,5,6]])
print(b.shape)                     # Print: (2, 3)
print(b[0, 0], b[0, 1], b[1, 1]) # Print: 1 2 5
```

Funkcje umożliwiające tworzenie predefiniowanych tablic:

```
# Create an array of all zeros
a = np.zeros((2,2))
print(a)               # Print: [[ 0.  0.]
#         [ 0.  0.]]
# Create an array of all ones
b = np.ones((2,2))
print(b)               # Print: [[ 1.  1.]]
#         [ 1.  1.]]
# Create a constant array
c = np.full((2,2), 9)
print(c)               # Print: [[ 9.  9.]
#         [ 9.  9.]]

# Create a 2x2 identity matrix
d = np.eye(2)
print(d)               # Print: [[ 1.  0.]
#         [ 0.  1.]]

# Create an array filled with random values
e = np.random.random((2,2))
print(e)
# Print: [[ 0.87230167  0.81443941]
#       [ 0.23624134  0.87236687]]
```

Tablice Numpy można dzielić na części:

```
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])
b = a[:2, 1:3]
print(b)        # Print: [[2 3],
#         [6 7]]

# b[0, 1] is a reference to a[0, 2]
print(b[0, 1])  # Print: 3
b[0, 0] = 9
print(a[0, 1])  # Print: 9
```

Indeksowanie tablicy z wartościami binarnymi:

```
a = np.array([[1,2], [3, 4], [5, 6]])

# Elements of a that are bigger than 2;
bool_idx = (a > 2)
print(bool_idx)      # Print: [[False False  True]
#         [ True  True  True]]

# We can use boolean array for indexing
print(a[bool_idx])  # Print: [3 4 5 6]
print(a[a > 2])     # Print: [3 4 5 6]
# We can use tuple for indexing
idx = ((0, 1), (1,2))
print(a[idx])       # Print: [2 6]
```

Podstawowe funkcje matematyczne działają elementarnie na tablicach:

```
x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum
print(x + y)
print(np.add(x, y))
# Result [[ 6.0  8.0]
```

```
#        [10.0 12.0]]
```

Zauważ, że w przeciwieństwie do MATLAB-a operacja $*$ jest mnożeniem przez elementy macierzy, a nie mnożeniem dwóch macierzy.

```
# Elementwise product
print(x * y)
print(np.multiply(x, y))
# Result [[ 5.0 12.0]
#        [21.0 32.0]]

# Elementwise division
print(x / y)
print(np.divide(x, y))
# Result [[ 0.2          0.33333333]
#        [ 0.42857143  0.5        ]]
```

Możemy zmienić kształt lub transpozycję macierzy:

```
x = np.array([[0, 1], [2, 3], [4, 5]])
print(x)
# Print: [[0, 1],
[2, 3],
[4, 5]])

y = np.reshape(a, (2, 3)) # C-like index ordering
print(y)
# Print: [[0, 1, 2],
[3, 4, 5]]
```

Broadcast to mechanizm pozwalający na pracę z tablicami o różnych kształtach podczas wykonywania operacji arytmetycznych.

```
x = np.array([[1,2,3],[4,5,6],[7,8,9],[10, 11, 12]])
v = np.array([1, 0, 1])
y = np.empty_like(x)

# Dodaj wektor v do kazdego wiersza macierzy x
for i in range(4):
y[i, :] = x[i, :] + v
print(y)
# Print [[ 2  2  4] [ 5  5  7] [ 8  8 10] [11 11 13]]
```

5.2 Biblioteka OpenCV

Biblioteka OpenCV została stworzona do przetwarzania i obróbki obrazów. Zawiera szereg metod związanych z podstawowymi metodami przetwarzania obrazów oraz klasycznym metodami wizji komputerowej, która stanowią często jeden z procesów preprocessingu do zastosowania w metodach uczenia maszynowego. Biblioteka umożliwiali wczytanie i zapis w wielu formatach obrazów graficznych. Istotnym elementem jest efektywna implementacja, która jest zazwyczaj znaczenie szybsza od innych bibliotek.

Wczytywanie i wyświetlanie obrazów:

```
import cv2
from google.colab.patches import cv2_imshow

image1 = cv2.imread(myfiles[0])
cv2_imshow(image1)
image2 = cv2.imread(myfiles[1])
cv2_imshow(image2)
```

Zapisywanie obrazów z określonym formatem graficznym:

```
% format png
new_image_path = '/content/result.png'
cv2.imwrite(new_image_path, image1)

% format jpg z okresleniem jakosci
new_image_path = '/content/result.jpg'
cv2.imwrite(new_image_path, image1, [int(cv2.IMWRITE_JPEG_QUALITY), 25])
```

Formatem typu danych obrazu w bibliotece OpenCV jest numpy array. W celu wyświetlenia wielu obrazów zalecane jest zastosowanie biblioteki matplotlib.pyplot. Biblioteka zawiera wiele funkcji pozwalających na wyświetlanie wykresów i rysunków z wieloma opcjami ich modyfikacji. Przed przystąpieniem do pracy nad wieloma plikami warto stworzyć listę ścieżek do poszczególnych plików zgodnie z zadanym formatem.

```
from os import listdir

search_jpg = lambda a, b: [a + '/'+ f for f in listdir(a) if f.endswith(b)]
fextension = '.jpg'
myfiles = search_jpg(mypath, fextension)
print(myfiles)
```

Wyświetlenie obrazów wczytanych za pomocą biblioteki openCV wymaga dodatkowych zmian w kodzie programu. Odczytane przez OpenCV obrazy wykorzystują przestrzeń barw BGR, a nie RGB. Różnica ta powoduje nie właściwe wyświetlenie składowych przestrzeni barw obrazu. Przykładowy kod programu z poprawnym wyświetleniem barw:

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

plt.figure()
for i in range(0, 4):
plt.subplot(2, 2, i+1)
im_bgr = cv2.imread(myfiles[i])
im_rgb = cv2.cvtColor(im_bgr, cv2.COLOR_BGR2RGB)
plt.imshow(im_rgb)
plt.show()
```

Przy pracy z pakietem Colab warto wiedzieć, że Funkcje cv2.imshow() i cv.imshow() z pakietu openCV są niezgodne z notatnikiem Jupyter, a przez to nie jest możliwe wyświetlenie obrazu z ich pomocą. Jako zamiennik możesz użyć funkcji:

```
import cv2
img = cv2.imread('logo.png', cv2.IMREAD_UNCHANGED)
cv2_imshow(img)
```

Więcej na temat biblioteki będzie można dowiedzieć się podczas laboratoriów wykonując zadania wymagające zastosowania konkretnych funkcji opisanych w instrukcji danego laboratorium

5.3 Matplotlib

Matplotlib to biblioteka niskiego poziomu do tworzenia wykresów w Pythonie, która służy jako narzędzie do wizualizacji. Umożliwia tworzenie szerokiej gamy wykresów, od prostych wykresów liniowych po skomplikowane wizualizacje 3D. Większość narzędzi Matplotlib znajduje się w module pyplot i zazwyczaj są importowane pod aliasem plt oraz stosowana z biblioteką Numpy:

```
import matplotlib.pyplot as plt
import numpy as np
```

Pyplot zapewnia interfejs podobny do MATLABa. Wykresy danych tworzymy za pomocą Figures, z których każdy może zawierać jedną lub więcej osi, a przez to określających układ współrzędnych. Wygląd jak i każdy parametr wykresu można zmienić za pomocą wielu dostępnych funkcji. Przykład tworzenia prostego wykresu funkcji sinus z etykietami osi i tytułem:

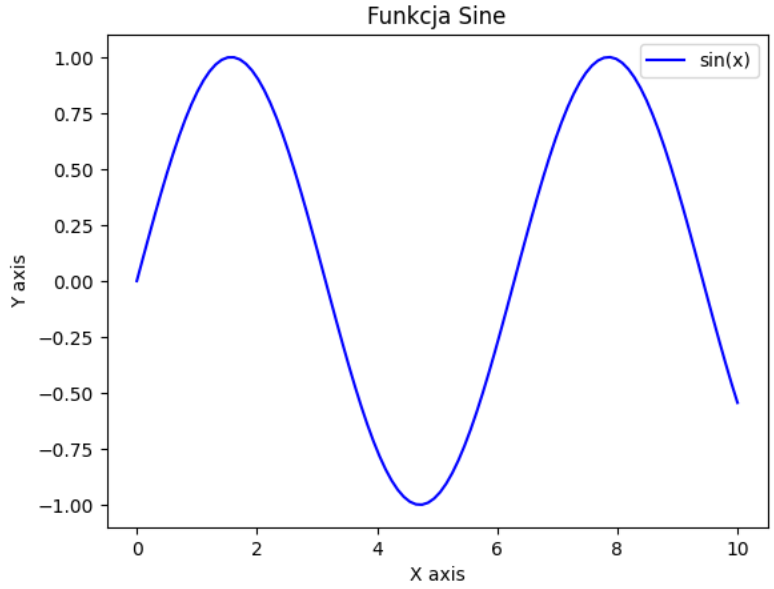
```
# Dane
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Tworzenie wykresu
plt.plot(x, y, label="sin(x)", color="blue")

# Opis osi i tytul
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.title("Funkcja Sine")

# Dodanie legendy
plt.legend()

# Wyswietlenie wykresu
plt.show()
```



Najprostszym sposobem utworzenia kilku rysunków w jednym oknie jest użycie pyplot.subplots. Przykład tworzenia złożonego wykresu z dwoma przebiegami w różnych subplotach oraz zaawansowanym opisem:

```
# Dane
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.exp(-0.5 * x) * np.sin(2 * x)

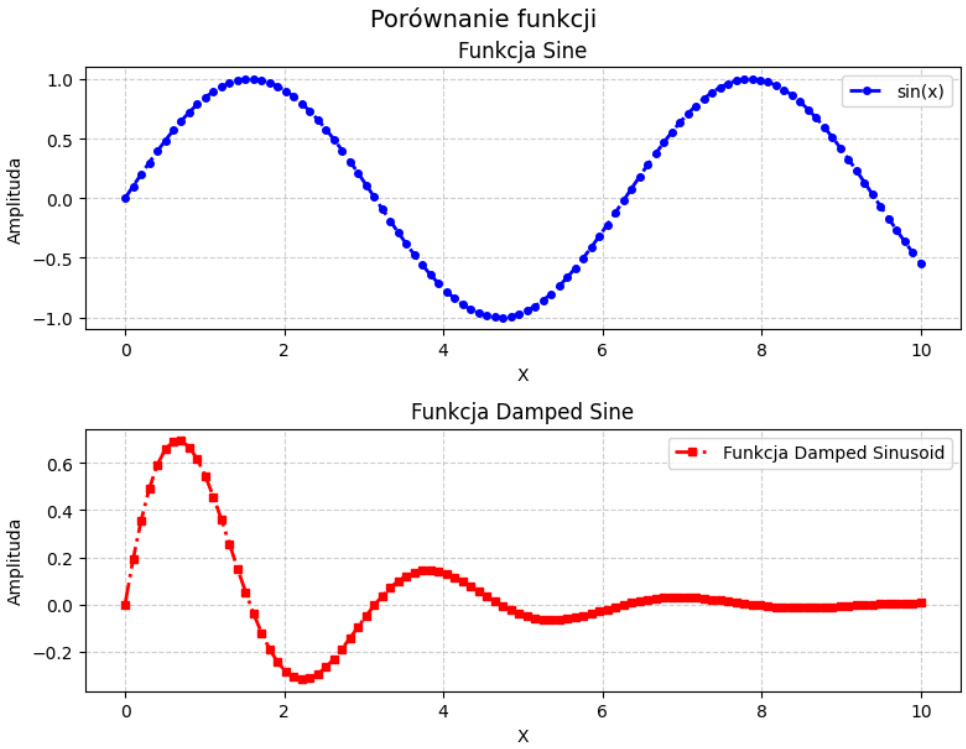
# Tworzenie figury i podzial na dwa subploty
fig, axs = plt.subplots(2, 1, figsize=(8, 6))

# Pierwszy subplot: funkcja sinus
axs[0].plot(x, y1, label="sin(x)", color="blue", linestyle="--", linewidth=2, marker="o",
    markersize=4)
axs[0].set_title("Funkcja Sine", fontsize=12)
axs[0].set_xlabel("X")
axs[0].set_ylabel("Amplituda")
axs[0].grid(True, linestyle="--", alpha=0.6)
axs[0].legend()

# Drugi subplot: tlumiona funkcja sinusoidalna
axs[1].plot(x, y2, label="Funkcja Damped Sinusoid", color="red", linestyle="-.", linewidth=2,
    marker="s", markersize=4)
axs[1].set_title("Funkcja Damped Sine", fontsize=12)
axs[1].set_xlabel("X")
axs[1].set_ylabel("Amplituda")
axs[1].grid(True, linestyle="--", alpha=0.6)
axs[1].legend()

# Dodatkowe dostosowania wygladu
plt.tight_layout()
plt.suptitle("Porownanie funkcji", fontsize=14, y=1.02)  # Tytul dla calej figury

# Wyswietlenie wykresu
plt.show()
```



Dodatkowo dwa wykresy pokazują różnicę w podejściu do tworzenia wykresów. Istnieją zasadniczo dwa sposoby korzystania z Matplotlib:

- Jawne tworzenie Figures i Axes oraz używanie obiektów pyplot do tworzenia wykresów („styl obiektowy").

```
x = np.linspace(0, 10, 100)  # Sample data.

fig, ax = plt.subplots(figsize=(5, 2.7))
ax.plot(x, x, label='linear')
ax.plot(x, x**2, label='quadratic')
ax.set_ylabel('y label')
ax.set_title("Plot")
ax.legend()
fig.show()
```

- Niejawne tworzenie Figures i Axes oraz używanie funkcji pyplot do tworzenia wykresów.

```
x = np.linspace(0, 10, 100)  # Sample data.

plt.figure(figsize=(5, 2.7))
plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadratic')
plt.ylabel('y label')
plt.title("Plot")
plt.legend()
plt.show(block=False)
```

Oba sposoby korzystania z Matplotlib pokażą te same wyniki, ale istnieją pewne różnice w zarządzaniu nimi.